Raphaël Hennocq
Youssef Khemira
Simon Trichard

NF22
Projet Excel VBA
Mastermind

P25

Bérangère BIHAN-AVALLE

Sommaire

I/ Rappel du cahier des charges et utilisateur type **3**
II/ Mode d'emploi **4**
a) Lancer l'application 4
b) Jouer une partie 4
c) Consulter les scores 6
III/ Retour bêta-testeur **6**
IV/ Algorithmes **7**
a) Partie de Youssef 7
b) Partie de Simon 8
c) Partie de Raphaël 10
V/ Discussion **11**
a) Contributions 11
i) Partie de Simon 11
ii) Partie de Raphaël 12
iii) Partie de Youssef 13
b) Voies d'améliorations 13

Dossier projet VB NF22 - Mastermind Excel VBA

Le but de ce projet était de créer une application en VBA (Visual Basic for Applications) à l'aide d'Excel. L'application est un jeu Mastermind implémenté grâce à des formulaires utilisateurs, les données étant stockées sur des feuilles de calculs et la logique réalisée en partie grâce à des formules de cellules.

I/ Rappel du cahier des charges et utilisateur type

Afin de réaliser le jeu Mastermind sur Excel, il a fallu tout d'abord étudier les règles du jeu. L'utilisateur dispose de 10 essais pour retrouver la combinaison de 4 couleurs dans l'ordre (le "code") choisie par son adversaire. Ici, les couleurs sont choisies aléatoirement afin que l'utilisateur puisse jouer seul. Il y a 6 couleurs : rouge, vert, bleu, jaune, orange, violet. Pour retrouver la combinaison gagnante, à chaque tour, le joueur dispose d'indices qui sont des pions de couleur noire ou blanche, placés dans une colonne séparée. En fonction des couleurs qu'il a indiqué, on fournit les indices suivants : si une des couleurs qu'il a indiqué est dans la combinaison mais au mauvais endroit, un pion blanc est placé ; si une couleur est bien placée, un pion noir est placé.

Le but du jeu est de réussir à trouver la combinaison en le moins d'essais possibles, et en moins de 10 essais. L'utilisateur peut donc saisir, pour chacun des 4 emplacements, une des six couleurs possibles. Il peut ensuite confirmer son choix. L'interface affiche la combinaison qu'il vient d'entrer, suivie des indices correspondants.

Une fois la partie terminée, son score est enregistré. Afin d'associer un score à un joueur, son nom et prénom lui sont demandés au préalable. La date du jour est aussi enregistrée pour établir des statistiques ultérieurement. Que ce soit une victoire ou une défaite, l'utilisateur a le choix entre afficher le tableau des scores ou rejouer une partie.

Le tableau des scores affiche tous les scores des parties qui ont été jouées, dans l'ordre chronologique. L'utilisateur peut double-cliquer sur un score : l'application récupérera alors tous les scores du joueur à partir de son nom et de son prénom, puis créera un graphique affichant l'évolution du score en fonction de la date de jeu.

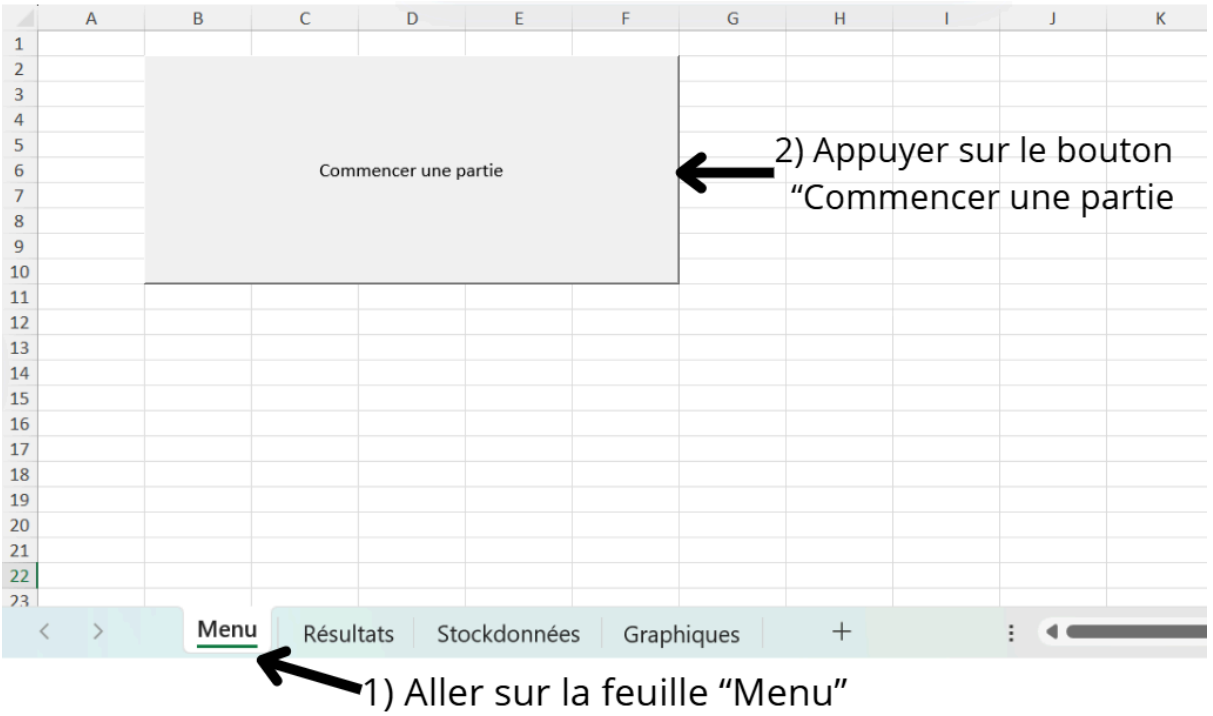
L'utilisateur peut également accéder directement au tableau des scores via le menu principal. Pour lancer le jeu, une feuille Excel intitulée Menu contient un gros bouton permettant d'accéder à l'interface du jeu.

L'application est conçue pour le grand public : grâce à son interface intuitive, enfants comme adultes peuvent s'en servir. C'est un jeu qui n'a pas d'âge et il n'y a pas de profil particulier d'utilisateur type.

II/ Mode d'emploi

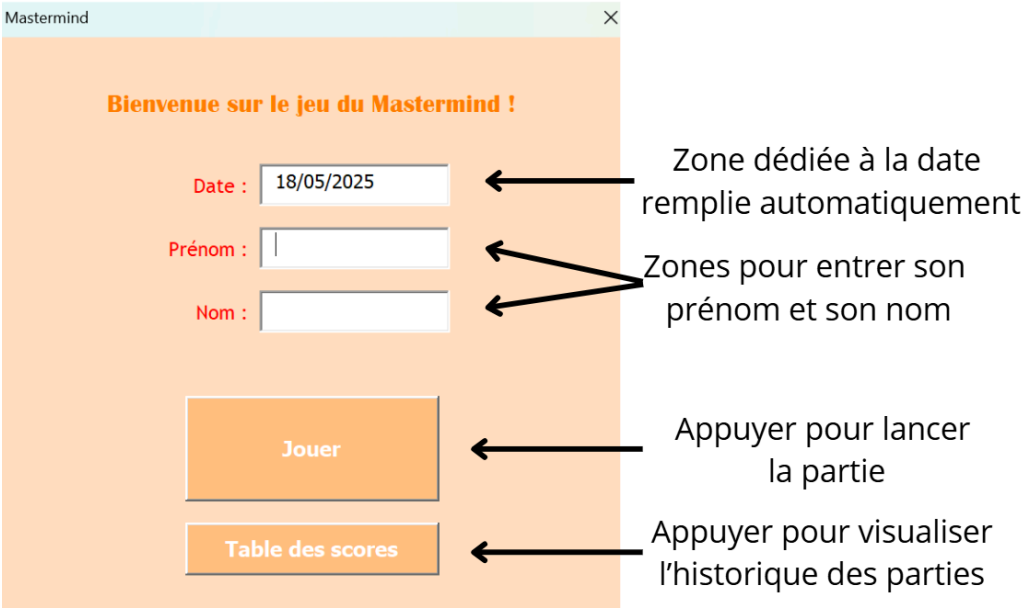
a) Lancer l'application

Pour lancer l'application, il faut cliquer sur le bouton "Commencer une partie" qui apparaît sur la feuille lors de l'ouverture du fichier Excel. La feuille est intitulée "Menu", c'est celle-là qu'il faut ouvrir pour trouver le bouton dans le cas où ce n'est pas la feuille active.

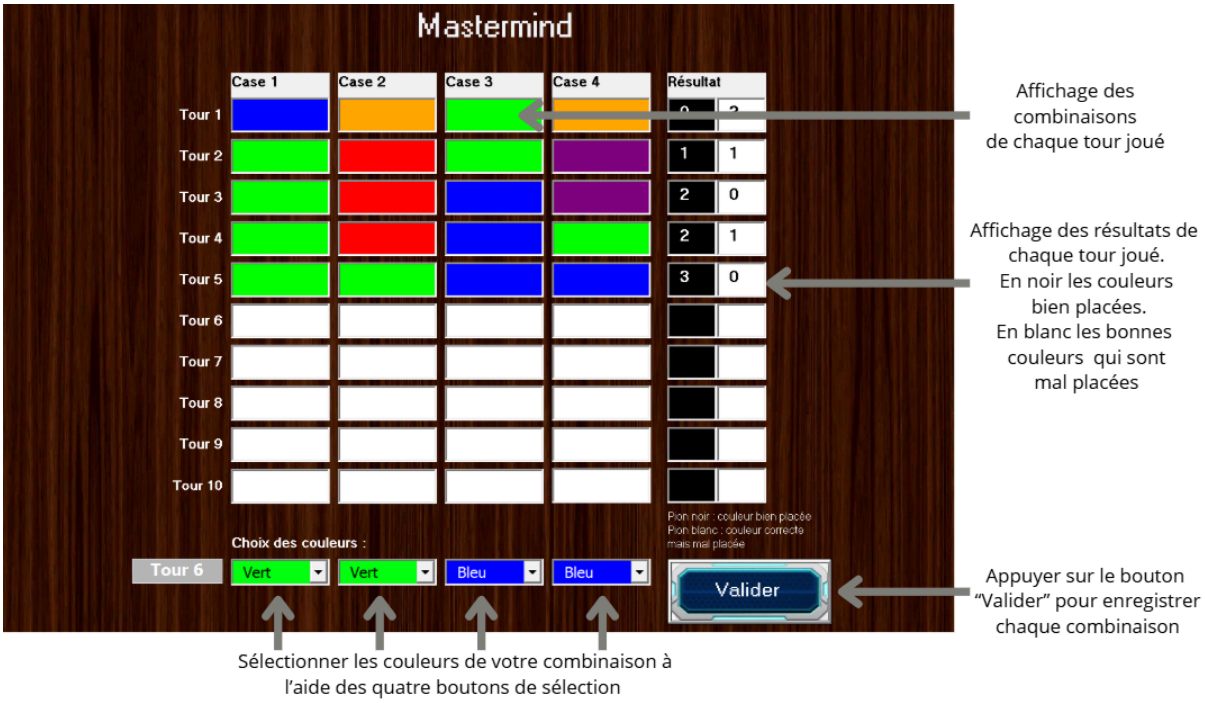


b) Jouer une partie

Une fois dans le menu principal, l'utilisateur peut entrer son nom et son prénom. Ces informations sont facultatives, mais si l'utilisateur ne les saisit pas, il ne pourra pas obtenir de statistiques personnalisées de l'évolution de son score au fil du temps. Une fois ces informations fournies, il suffit de presser le bouton "Jouer". La fenêtre de jeu s'affiche alors.

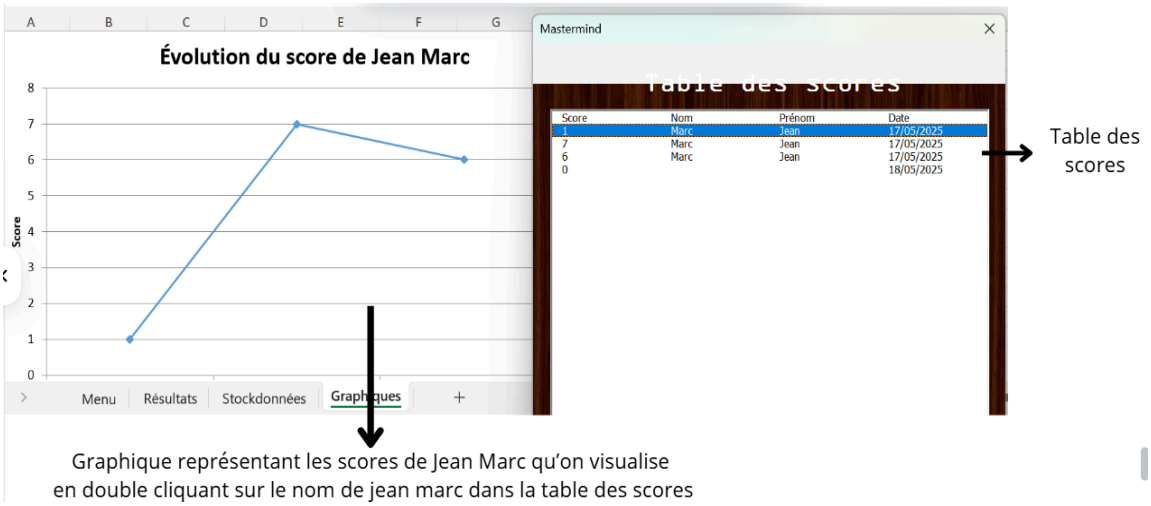


L'interface est composée d'un tableau principal de 10 lignes et 4 colonnes. Chaque ligne est un tour, et affiche la tentative du tour associé. À droite de ces lignes, on retrouve 2 colonnes d'indices qui indiquent les pions noirs et blancs de chaque tour. En bas du tableau, on retrouve 4 boutons de sélection qui permettent de choisir les couleurs de la tentative. Lorsque l'on choisit une couleur, le bouton de sélection prend cette couleur, ce qui permet de prévisualiser la combinaison que l'on entre. Une fois cette combinaison choisie, on presse le bouton "Valider" puis la tentative est enregistrée. La partie se termine lorsque l'utilisateur trouve la combinaison gagnante ou lorsqu'il épuise ses 10 tentatives. Le score associé est alors inscrit.



c) Consulter les scores

Pour consulter le score, l'utilisateur doit juste presser un bouton "Table des scores". Ces boutons sont affichés sur le menu principal et sur la page de fin de jeu, une fois que l'utilisateur a gagné ou perdu.



III/ Retour bêta-testeur

Voici les points d'amélioration et de correction relevés lors de la phase de bêta-testing :

Il était difficile d'interpréter correctement la combinaison que l'on était en train de saisir. Une fonction permettant de changer la couleur des ComboBox a alors été implémentée, ce qui permet de mieux se rendre compte de ce que l'on est en train de proposer.

La table des scores est utile mais il manquait un moyen de consulter efficacement les scores qui nous concernent, surtout lorsqu'il commence à avoir beaucoup de scores. Pour corriger cela, une fonction de statistiques personnelles a été ajoutée : en double cliquant sur une ligne nous concernant, on affiche directement l'évolution de notre score au fil des parties jouées.

D'une manière globale, le testeur a aussi permis de relever des bugs et failles dans le code de l'application, en testant des manipulations que nous n'avions pas faites. Le fait de perdre plusieurs fois d'affilée par exemple, entraînait un bug lié à l'affichage des différentes pages, notamment la page dédiée au résultat de la partie lorsque l'utilisateur perdait.

IV/ Algorithmes

a) Partie de Youssef

Voici l'algorithme en pseudo-code correspondant à la procédure permettant de générer un graphique de l'évolution du score d'un joueur en fonction de la date, ainsi que les explications associées :

```
Procédure GénérerGraphique(nom [chaîne de caractères], prénom [chaîne de caractères])
Début
    feuilleRésultats ← Feuille de calcul "Résultats"
    feuilleGraphiques ← Feuille de calcul "Graphiques"
    ligne ← 3
    compteur ← 0

    // Nettoyer la feuille des graphiques et ajouter les en-têtes
    Effacer le contenu des cellules A1 à B1000 de feuilleGraphiques
    feuilleGraphiques[A1] ← "Date"
    feuilleGraphiques[B1] ← "Score"

    // Parcourir les données de la feuille Résultats
    Tant que feuilleRésultats[ligne, 2] ≠ vide Faire
     Si feuilleRésultats[ligne, 3] = nom ET feuilleRésultats[ligne,
     4] = prénom Alors
    compteur ← compteur + 1
    feuilleGraphiques[compteur + 1, 1] ← feuilleRésultats[ligne, 5]
    feuilleGraphiques[compteur + 1, 2] ← feuilleRésultats[ligne, 2]
     FinSi
     ligne ← ligne + 1
    FinTantQue

    // Vérifier s'il y a des données à afficher
    Si compteur = 0 Alors
        Afficher "Aucune donnée trouvée pour " & prénom & " " & nom
        Sortir de la procédure
    FinSi

    // Supprimer les anciens graphiques
    Pour chaque graphique dans feuilleGraphiques Faire
        Supprimer graphique
```

```
    FinPour

    // Créer le nouveau graphique
    nouveauGraphique ← Ajouter un graphique dans feuilleGraphiques à
la position (0, 0), taille (500, 300)
    plageDonnées ← feuilleGraphiques[“A1:B” & (compteur + 1)]

    // Configurer le graphique
    Définir le type du graphique comme Courbe avec marqueurs
    Définir la source des données comme plageDonnées
    Ajouter un titre : "Évolution du score de " & prénom & " " & nom
    Ajouter un titre à l’axe horizontal : "Date"
    Ajouter un titre à l’axe vertical : "Score"

    // Afficher la feuille contenant le graphique
    Activer feuilleGraphiques
Fin
```

Pour générer un graphique, on a d’abord besoin de récupérer tous les scores du joueur. Pour cela, on conserve le nom et prénom du joueur sur lequel on a double cliqué. On parcourt ensuite le tableau des scores et on conserve chaque score et date à chaque fois que le nom et prénom du joueur est le même que celui passé en argument. Cette étape correspond au parcours du tableau des scores et de la vérification pour chaque ligne : `Si feuilleRésultats[ligne, 3] = nom ET feuilleRésultats[ligne, 4] = prénom Alors`. Si la condition est remplie, on ajoute le score et la date à un tableau temporaire créé directement sur la feuille Graphiques. Ce tableau a été préalablement affecté d’une entête contenant une colonne Score et une colonne Date. C’est à partir de ce tableau que l’on trace par la suite le graphe de l’évolution du score selon la date de la partie. On vérifie qu’il y a au moins 1 score qui a été récupéré. En principe, il y aura au moins une ligne puisque l’on a double cliqué sur un score pour lancer la macro. C’est une vérification supplémentaire pour prévenir de tout bug.

On nettoie ensuite tous les graphes déjà présents sur la feuille Graphiques, puis on crée un nouveau graphique à partir du tableau temporaire. Enfin, on affiche la feuille Graphiques.

b) Partie de Simon

Présentation de l’algorithme qui s’effectue à chaque fois que l’utilisateur clique sur le bouton “Vérifier” se concentrant sur la combinaison entrée par l’utilisateur.
Variable :

```
        Début :  Tour, réelle
        Intermédiaire : i, réelle
Début
Tour ← Tour + 1
i ← 1
Tant que i<5
Prendre la valeur de l'objet ("couleur" & i)
Si "couleur" & i = "Rouge"
        L'objet ("Case" & ((Tour - 1) * 10 + i)) prend pour valeur la couleur rouge
        cellule((2 + Tour), (i + 1)) ← 1
Si "couleur" & i = "Rouge"
        L'objet ("Case" & ((Tour - 1) * 10 + i)) prend pour valeur la couleur vert
        cellule((2 + Tour), (i + 1)) ← 2
Si "couleur" & i = "Vert"
        L'objet ("Case" & ((Tour - 1) * 10 + i)) prend pour valeur la couleur bleu
        cellule((2 + Tour), (i + 1)) ← 3
Si "couleur" & i = "Bleu"
        L'objet ("Case" & ((Tour - 1) * 10 + i)) prend pour valeur la couleur jaune
        cellule((2 + Tour), (i + 1)) ← 4
Si "couleur" & i = "Violet"
        L'objet ("Case" & ((Tour - 1) * 10 + i)) prend pour valeur la couleur violet
        cellule((2 + Tour), (i + 1)) ← 5
Si "couleur" & i = "Orange"
        L'objet ("Case" & ((Tour - 1) * 10 + i)) prend pour valeur la couleur orange
        cellule((2 + Tour), (i + 1)) ← 6
i ← i +1
Fin tant que
Fin
```

Cet algorithme a pour objectif d'assurer le remplissage en couleur des TextBoxs qui informent les combinaisons choisis pour les différents tours avec des couleurs, ainsi que le remplissage de la feuille "Stockdonnées" avec les valeurs attribuées à chaque couleur (1 pour le rouge par exemple).
La principale difficulté rencontrée dans la conception de cet algorithme est liée à la gestion d'un grand nombre de TextBox et de cellules à renseigner, dont le nombre peut varier de 4 à 40. Pour limiter le nombre de lignes de code et de variables à intégrer à l'algorithme, j'ai pensé à nommer d'une manière réfléchie les ComboBoxs et le TextBoxs. Les TextBoxs ayant pour mission de renvoyer les combinaisons de couleur choisit pour les tours joués se nomment toutes avec la forme "Casexy" avec x indiquant le numéros de ligne (première ligne pour le premier tour, deuxième ligne pour le deuxième tour…) puis le y pour le numéros de la couleur dans la combinaisons (y=3 pour la troisième couleur d'une même combinaison). Les ComboBoxs qui permettent à l'utilisateur de choisir les couleurs d'une combinaison se nomment toutes avec la forme "couleury" avec ici encore le y pour le numéros de

la couleur dans la combinaison. Finalement seulement deux variables suffisent à la réalisation de l'algorithme qui sont "Tour" et "i". A chaque fois que l'utilisateur clique sur valider, la variable Tour augmente de 1 ce qui permet de remplir les TextBoxs et les cellules de la ligne suivante. De plus une boucle qui se sert de la variable i pour tourner permet de remplir les quatre TextBoxs et cellules d'une même ligne.

c) Partie de Raphaël

On va développer la partie du code lorsque la partie est gagnée ou perdue. Le code est le suivant :

```
If Cells(2 + Tour, 7).Value = 4 Then 'Vérifier si on a les 4 couleurs bien placées
        Worksheets("Résultats").Activate
        Cells.Find(what:="score").Activate
        l = ActiveCell.Row
        c = ActiveCell.Column
        While Cells(l, c).Value <> ""
            l = l + 1
        Wend
        Cells(l - 1, c).Value = 12 - Tour 'afficher dans la colonne score, le score associé qui est 12-(le nombre de tour)
        Me.Hide 'Cacher la Form de la partie
        FormVictoire.Show 'Afficher la Form lorsqu'on gagne la partie
ElseIf Cells(2 + Tour, 7).Value < 4 And Tour = 11 Then 'Vérifier si au dernier tour on a pas trouvé les 4 bonnes positions
        Me.Hide 'Cacher la Form de la partie
        FormDefaite.Show 'Afficher la Form lorsqu'on perd la partie
End If
```

L'écriture algorithmique est développée ci-dessous :

Variable :
    Entrée : nbr_Tour, pion_correct
    Sortie : score

Début
    SI pion_correct ← 4 ALORS
        score ← 12 - nbr_Tour
        Lire ("Bravo, vous avez gagné la partie, votre score est", score)
    SI pion_correct < 4 ET nbr_Tour ← 11 ALORS
        score ← 0
        Lire ("Dommage, vous n'avez pas trouvé le code assez vite")
Fin

J'ai choisi d'écrire cette partie de code en algorithme pour montrer la différence entre l'écriture algorithmique qui est très rapide est simplifiée et l'écriture en VBA qui contient plus de détails.

Nous remarquons que la principale différence est que dans le code nous avons besoin d'aller chercher des cellules de feuille de calcul. En effet, les valeurs "pion_correct" sont dans la feuille "Stockdonnées". Lorsque nous voulons afficher le score à la fin de la partie, nous devons retourner dans la feuille "Résultat" puis rechercher la colonne "score" et défiler jusqu'à trouver la première cellule vide puis

inscrire le nouveau score dans la ligne d'au-dessus (le score était initialement à 0). Alors que pour la partie algorithmique, il suffit d'incrémenter la bonne valeur à la variable score.

Lorsque la partie est perdue, nous remarquons que nous devons incrémenter, en algorithme, la variable score à 0 alors que dans la partie code, il n'y a rien de tel. Cela s'explique par le fait qu'en VBA, le score est déjà incrémenté initialement à 0 au cas où l'utilisateur quitterait la partie en cours de jeu.

Enfin, la dernière différence notable est qu'en algorithme, nous utilisons de simple MessageBox pour prévenir de la défaite ou de la victoire, alors qu'en VBA, ce sont de nouveaux formulaires qui s'affichent.

V/ Discussion

a) Contributions

Durant ce projet, nous avons tous contribué à chaque partie du jeu. Dans les faits, Simon et Raphaël ont commencé à coder l'ensemble du jeu et Youssef a résolu chaque bug auquel Simon et Raphaël ont fait face et a amélioré le code afin de le rendre plus fluide et plus agréable.

i) Partie de Simon

Pour ce projet Excel, je me suis surtout focalisé sur le jeu mastermind en lui même. J'ai réalisé la feuille intitulée "Stockdonnées", en commençant par trouver une manière de générer aléatoirement une combinaison de quatre couleurs. Il est rapidement devenu évident que les données stockées sur la feuille concernant la combinaison secrète serait des chiffres et non des couleurs puisque j'ai choisi la formule "ALEA.ENTRE.BORNES(1;6)" appliquer sur les cellules [B2;E2] pour tirer au sort la combinaison. Cependant, à chaque modification de la feuille cette formule change, il a donc fallu bloquer une formule pour chaque nouvelle partie, ce que j'ai réussi à faire avec une macro qui copier-coller les valeur numérique des cellules [B3;E3]. J'ai décidé que les formules entrées par l'utilisateur seraient stockées et analysées dans les cellules allant de [B4;E4] à [B13;E13]. J'ai ensuite mis au point une deuxième formule qui permet d'analyser les réponses correctes, c' est une formule à condition "Si" qui compare la réponse donnée en fonction de son ordre (de première jusqu'à quatrième) à la combinaison secrète. La formule additionne les bonnes réponses et peut donc aller de 0 à 4, elle est insérée dans les cellules [G4;G13]. Youssef s'est chargé de trouver la solution permettant d'analyser le nombre de bonnes couleurs mal placées, on a ensuite inséré cette solution dans les cellules [H4;H13].

Après avoir réalisé l'architecture de la feuille excel qui permet d'analyser les réponses données par l'utilisateur j'ai réalisé un formulaire qui permet à l'utilisateur

de soumettre des combinaisons et d'afficher des retours pour celle-ci. Ce formulaire se nomme "FormPartie" et possède plusieurs éléments : un label qui permet d'afficher le nombre de tour, des ComboBoxs qui permettent de choisir les différentes couleurs d'une combinaison, de TextBoxs dont certaines renvoie les combinaisons de couleurs choisis pour chaque tour joué (ces TextBoxs ont étaient facilement incorporées à la suite des retours des beta-testeurs) et d'autres renvois les résultats de chaque combinaison ainsi qu'un bouton pour valider le choix d'une combinaison.

L'action la plus importante que j'ai eu à mettre en place sur ce formulaire est la validation d'une combinaison qui se fait en appuyant sur le bouton "Valider". Lorsqu'une combinaison est valider par l'utilisateur, plusieurs actions doivent être effectué : le remplissage de la feuille "Stockdonnées" avec les valeurs attribuées à chaque couleur (1 pour le rouge par exemple), le remplissage des TextBox qui informe des combinaisons choisis pour les différents tours avec des couleurs et enfin le remplissage des TextBoxs qui donne le résultat associé à la combinaison joué en allant chercher l'information depuis la feuille "Stockdonnées".

ii) Partie de Raphaël

Comme expliqué précédemment, nous avons tous contribué à chaque partie. Je développerai donc mes travaux dans l'ordre du jeu.

Sur le premier formulaire qui s'intitule "FormDebut", j'ai fait en sorte que le premier caractère du Nom et du Prénom de l'utilisateur soit en majuscule. Cela permet à l'utilisateur, qui peut oublier la majuscule, de garder un suivi de ses scores. Le bouton play est, lui, très important en termes de code qu'il contient. En effet, lorsqu'on clique dessus, il ajoutera, dans un premier temps, le nom, le prénom de l'utilisateur ainsi que la date dans la feuille de calcul "Résultat" et le score est initialisé à 0. Nous sommes obligés de l'initialiser à 0 car si le joueur quitte la partie avant la fin du jeu, la colonne score doit avoir une valeur afin d'éviter tous soucis avec le graphique. Ensuite, ce bouton permet aussi d'aller sur la feuille "Stockdonnées" et d'initialiser la cellule "Tour" à 1 car le jeu commence au premier tour par définition. Toujours sur la même feuille, on efface l'historique de la partie précédente sinon la colonne "à revoir" gardera surement une valeur de 4 et le jeu s'arrêtera automatiquement sur une victoire.

Ensuite, vient le formulaire FormPartie qui correspond à l'affichage du jeu. En appuyant sur le bouton "Valider", on augmente la valeur de la cellule "Tour" de 1, on incrémente la variable "Tour" à la valeur de cette cellule et on modifie le label nous indiquant à quel tour nous sommes au tour suivant. A la fin de cette procédure, nous trouvons une boucle "Si". Cette boucle permet de vérifier à chaque tour si la partie est gagnée ou pas. Pour ce faire, elle regarde si dans la colonne "correct" on a la valeur 4 qui correspondrait aux 4 couleurs placées correctement. Si c'est le cas, on va dans la colonne "score" de la feuille "Résultats". On descend de ligne jusqu'à la première cellule vide puis on met le score dans la ligne au-dessus (on rappelle que notre cellule n'était pas vide puisqu'elle valait 0 par défaut) et on affiche le formulaire

de victoire. Si, au contraire, on a pas trouvé le code en 10 coups (la variable “Tour” vaut 11 car en appuyant sur valider on ajoute plus 1), on laisse le score à 0 et le formulaire de défaite s’ouvre.

Les formulaires de victoire et de défaite (FormVictoire et FormDefaite) permettent uniquement de rejouer ou d’afficher le formulaire des scores. En rejouant, le formulaire de début de partie s’affiche à nouveau.

iii) Partie de Youssef

Pour ma part du projet, j’ai laissé Raphaël et Simon poser les bases du jeu. Je me suis tout d’abord occupé à réaliser une macro dans un module qui permet de compter le nombre de couleurs correctes dans la combinaison mais mal placées. Cette fonction ne pouvait pas être réalisée à l’aide d’une formule Excel contenue dans une seule case car elle est trop complexe et nécessite de retenir certaines informations afin de respecter les règles officielles du jeu. Il était donc plus simple de l’implémenter en VBA et de s’en servir directement dans une cellule, sachant que la méthode de fonctionnement établie était d’avoir toutes les informations sur l’état du jeu directement dans la feuille Excel, afin de n’avoir quasiment que de l’affichage à gérer avec les UserForms en VBA.

Une fois le jeu fonctionnel, j’ai refait toute l’interface du jeu au propre, pour chaque page du jeu, en modifiant les propriétés telles que la police, les couleurs, la cohérence du placement des éléments, l’ajout d’images de fond, etc. J’ai ensuite ajouté un nouveau UserForm pour pouvoir visualiser les scores des joueurs directement via le formulaire plutôt que de devoir regarder sur la feuille de calcul.

J’ai corrigé les bugs que j’ai trouvés, épuré et simplifié le code lorsque cela était possible et renommé les différents modules et userForms. Enfin, j’ai aussi implémenté les éléments suivants :

- Ajout d’une fonction qui modifie la couleur des comboBox permettant le choix de la combinaison, afin de rendre le jeu plus intuitif et faciliter la comparaison avec les essais précédents ;
- Remplissage automatique de la date dans le menu et verrouillage de la textBox ;
- Macro qui permet d’afficher l’évolution du score d’un joueur, en générant un graphe sur la feuille Graphiques lorsque l’on double clique sur une ligne concernant un score. Cette macro reprend le nom et le prénom du joueur de la ligne, cherche toutes ses autres parties, crée un tableau temporaire et trace un graphe de l’évolution de son score selon la date.

b) Voies d’améliorations

Il existe diverses voies d’améliorations pour rendre le jeu meilleur. L’interface pourrait être encore améliorée, même si les formulaires en VBA ont des limites. On pourrait aussi imaginer avoir d’autres modes de jeu dans le menu, par exemple une partie contre la montre, ou bien une partie avec des essais illimités, ou encore avec

plus de 4 couleurs (8 couleurs par exemple)... L'accès au formulaire du jeu pourrait aussi être amélioré, et les feuilles de calcul optimisées. L'affichage des scores pourrait être plus détaillé, et l'affichage des graphes pourrait éventuellement se faire directement via le formulaire, même si cela ajoute une complexité qui n'a pas été vue en cours.